Command Design Pattern

Intent

Turn a **request into an object**. Instead of calling a method directly, you wrap "the thing to do" in a command object that knows *which* receiver to act on and *what* to do to it. Because the request is now an object, you can pass it around, store it, queue it, log it, undo it, or hand it to something that fires it later without knowing what it does.

Here the requests are **workcell actions** (engage an actuator, disengage an actuator, grip a payload, move a joint), the things being controlled are **actuators** (a `JointActuator`, a `GripperActuator`), and the **`TaskQueue`** dispatches a pending task without knowing anything about the actuator behind it.

UML class diagram

```
 <<interface>> IActuatorCommand   <<interface>> IActuator
 | +run() |                      | +engage() +disengage() |
    ^   ^   ^   ^                    ^                ^
    |   |   |   |                    |                |
Engage Disengage GripPayload MoveJoint          JointActuator  GripperActuator
 (IActuator) (IActuator) (GripperActuator) (JointActuator)   +moveToNextWaypoint +gripPayload
    \       |        |        /
     bound receiver + action
        +----------------+
        |   TaskQueue    |  (invoker)
        | -pendingTask   |
        | +assignTask()  |
        | +dispatchTask()|--> task.run() --> actuator method
        +----------------+
```

The four roles (this pattern is defined by them)

```
controller/IActuatorCommand                the COMMAND     – the request interface: run()
controller/concretes/EngageActuatorCommand  CONCRETE COMMANDS – each binds a receiver + an action
                  /DisengageActuatorCommand
                  /GripPayloadCommand
                  /MoveJointCommand
receiver/IActuator (+ concretes/JointActuator, GripperActuator)  the RECEIVER – the object that does the real
work
invoker/TaskQueue                           the INVOKER     – holds a task and dispatches it
CommandDesignPattern (main)                 the CLIENT      – wires actuators, commands, invoker
```

- **IActuatorCommand** — a common interface with a single `run()`.
- **Concrete Command** — a small class that remembers *which receiver* and *which method* to call, and does so in `run()`.
- **Receiver** — the object that actually knows how to perform the work (the joint, the gripper).
- **Invoker** — dispatches a task (`dispatchTask()`) without knowing what the task does or who the receiver is.
- **Client** — creates the receivers and commands and assigns tasks to the invoker.

The code, line by line

`IActuatorCommand` — the request interface

```
public interface IActuatorCommand {
    public void run();
}
```

- One method: **`run()`** — "do the thing." Every request in this system is represented as an object implementing this interface.
- This is what makes a request first-class: anything that can hold an `IActuatorCommand` can trigger *any* action uniformly, without a giant `switch` over action types.

`IActuator`, `JointActuator`, `GripperActuator` — the receivers

```
public interface IActuator {
    void engage();
    void disengage();
}

public class JointActuator implements IActuator {
    @Override public void engage()    { System.out.println("Joint actuator is now engaged"); }
    @Override public void disengage() { System.out.println("Joint actuator is now disengaged"); }
    public void moveToNextWaypoint()  { System.out.println("Joint moved to next waypoint"); }
}

public class GripperActuator implements IActuator {
    @Override public void engage()    { System.out.println("Gripper actuator is now engaged"); }
    @Override public void disengage() { System.out.println("Gripper actuator is now disengaged"); }
    public void gripPayload()         { System.out.println("Jaws gripped around payload"); }
}
```

- The **receivers contain the actual behavior** — they know how to engage, move to a waypoint, grip a payload. The commands don't *do* the work; they *delegate* to a receiver that does.
- `engage()` / `disengage()` are shared across all actuators, so they live in the **`IActuator` interface**. `moveToNextWaypoint()` (joint-only) and `gripPayload()` (gripper-only) are actuator-specific, so they live only on the concrete classes. This split matters for how the commands are typed (below).

The concrete commands

```
public class EngageActuatorCommand implements IActuatorCommand {
    private IActuator actuator;
    public EngageActuatorCommand(IActuator actuator) { this.actuator = actuator; }
    @Override public void run() { actuator.engage(); }
}

public class DisengageActuatorCommand implements IActuatorCommand {
    private IActuator actuator;
    public DisengageActuatorCommand(IActuator actuator) { this.actuator = actuator; }
    @Override public void run() { actuator.disengage(); }
}

public class GripPayloadCommand implements IActuatorCommand {
    private GripperActuator gripper;
    public GripPayloadCommand(GripperActuator gripper) { this.gripper = gripper; }
    @Override public void run() { gripper.gripPayload(); }
}

public class MoveJointCommand implements IActuatorCommand {
    private JointActuator joint;
    public MoveJointCommand(JointActuator joint) { this.joint = joint; }
    @Override public void run() { joint.moveToNextWaypoint(); }
}
```

Each concrete command is the same tiny shape, and that shape *is* the pattern:

- **It stores a reference to its receiver** (an `IActuator`, `GripperActuator`, or `JointActuator`) — this is the "who to act on," captured at construction time.
- **Its `run()` calls one method on that receiver** — this is the "what to do." The command is essentially a bound (receiver, action) pair frozen into an object.
- **Notice the receiver types differ on purpose:**
- `EngageActuatorCommand` / `DisengageActuatorCommand` take the **`IActuator` interface** — because *every* actuator can be engaged/disengaged, so these commands work polymorphically with a joint, a gripper, or any future actuator.
- `GripPayloadCommand` takes a **`GripperActuator`** and `MoveJointCommand` takes a **`JointActuator`** — because those actions exist only on those concrete receivers, so the command is typed to the receiver that actually offers the method.

`TaskQueue` — the invoker

```
public class TaskQueue {
    private IActuatorCommand pendingTask;

    public void assignTask(IActuatorCommand task) { this.pendingTask = task; }

    public void dispatchTask() {
        if (pendingTask != null) {
            pendingTask.run();
        } else {
            System.out.println("No task assigned");
        }
    }
}
```

- `assignTask(...)` — you load a task into the queue. The queue holds it **by the `IActuatorCommand` interface**, so it can hold *any* task.
- `dispatchTask()` — the invoker's trigger. It just calls `pendingTask.run()`. Critically, the queue has **no idea** whether it's engaging a joint or gripping a payload — it only knows "I have a task; run it." That ignorance is the entire benefit (see *Why* below).
- The `null` guard makes dispatching with nothing assigned harmless ("No task assigned") instead of a `NullPointerException`.

`CommandDesignPattern` — the client

```
JointActuator joint = new JointActuator();
GripperActuator gripper = new GripperActuator();

IActuatorCommand engageJoint    = new EngageActuatorCommand(joint);
IActuatorCommand gripPayload    = new GripPayloadCommand(gripper);
IActuatorCommand moveJoint      = new MoveJointCommand(joint);
IActuatorCommand disengageJoint = new DisengageActuatorCommand(joint);

TaskQueue taskQueue = new TaskQueue();

taskQueue.assignTask(engageJoint);    taskQueue.dispatchTask();
taskQueue.assignTask(gripPayload);    taskQueue.dispatchTask();
taskQueue.assignTask(moveJoint);      taskQueue.dispatchTask();
taskQueue.assignTask(disengageJoint); taskQueue.dispatchTask();
```

- The **client does the wiring**: it creates the receivers, builds commands that bind actions to those receivers, and assigns tasks to the invoker.
- Then the **same** `dispatchTask()` produces four different behaviors, depending only on which task is currently loaded.

Console output:

```
Command Design Pattern
Joint actuator is now engaged
Jaws gripped around payload
Joint moved to next waypoint
Joint actuator is now disengaged
```

Why the design decisions

Why turn a method call into an object at all?

A direct call — `joint.engage()` — happens *right now* and can't be stored, passed, or reasoned about. Wrapping it as an object (`new EngageActuatorCommand(joint)`) makes the request something you can **hold and manipulate**: put it in a variable, store it in a list, queue it, log it, schedule it, or attach `undo()` to it later. Everything Command enables (undo/redo, macros, queues, transactional replay) flows from this one idea: *the request is now data.*

Why does the invoker hold an `IActuatorCommand` and not the receiver?

To **decouple the trigger from the work.** `TaskQueue` knows nothing about joints or grippers — it depends only on the `IActuatorCommand` interface. That means: - the same queue can dispatch *any* action, present or future, and - adding a new action (e.g. `HoldPositionCommand`) requires **zero changes** to `TaskQueue`.

If the queue called `joint.engage()` directly, it would be welded to the joint and would need editing for every new actuator or action. Command breaks that coupling: the invoker fires requests without knowing what they are.

Why does each command store its receiver?

Because a command must know *what to act on* when it's eventually dispatched — possibly long after it was created, by an invoker that has no reference to the receiver. Binding the receiver into the command at construction time makes the command **self-contained**: `run()` needs no arguments and no external context. That self-containment is what lets you queue or defer commands.

Why do the commands accept the receiver by *interface* where possible?

`EngageActuatorCommand(IActuator)` accepts the interface so one command class works for *every* actuator — engaging is universal, so there's no reason to tie the command to `JointActuator`. `GripPayloadCommand(GripperActuator)` accepts the concrete type only because `gripPayload()` isn't part of `IActuator`. The rule: **depend on the narrowest type that still exposes the method you need** — the interface when the action is shared, the concrete class when the action is specific.

Why the `null` check in `dispatchTask()`?

Defensive robustness. An invoker can exist with no task loaded (a freshly-built queue, or a slot that was never programmed). Handling that case gracefully ("No task assigned") is friendlier than crashing, and it documents that a task is optional state on the invoker.

Execution flow (the demo)

```
main (client wires everything)
  │
  ├── taskQueue.assignTask(engageJoint)      invoker now holds EngageActuatorCommand(joint)
  ├── taskQueue.dispatchTask()
  │         └── task.run()  →  EngageActuatorCommand.run()  →  joint.engage()          → "Joint actuator is now
engaged"
  │
  ├── taskQueue.assignTask(gripPayload)      invoker now holds GripPayloadCommand(gripper)
  ├── taskQueue.dispatchTask()
  │         └── task.run()  →  GripPayloadCommand.run()  →  gripper.gripPayload()      → "Jaws gripped around
payload"
  │
  ├── taskQueue.assignTask(moveJoint)        invoker now holds MoveJointCommand(joint)
  ├── taskQueue.dispatchTask()
  │         └── task.run()  →  MoveJointCommand.run()  →  joint.moveToNextWaypoint()   → "Joint moved to next
waypoint"
  │
  └── taskQueue.assignTask(disengageJoint)   invoker now holds DisengageActuatorCommand(joint)
      taskQueue.dispatchTask()
              └── task.run()  →  DisengageActuatorCommand.run()  →  joint.disengage() → "Joint actuator is now
disengaged"
```

The invoker does the identical thing every time (`task.run()`); the varying behavior comes entirely from *which command object* is loaded — and the invoker never learns what any of them actually do.

Notes / possible extensions (not changed in the code)

- **Undo/redo.** The classic Command extension is adding `undo()` to the interface; each command remembers enough to reverse itself (often by holding a **Memento** of the receiver's prior state). A history stack of dispatched commands then gives multi-level undo.
- **Macro commands.** A `MacroCommand` that holds a `List<IActuatorCommand>` and calls `run()` on each lets you compose several requests into one — a "pick-and-place" sequence that engages the gripper, moves the joint, and grips the payload in a single dispatch.
- **Queuing / logging / scheduling.** Because commands are objects, an invoker can push many onto an actual FIFO queue, run them on a worker thread, persist them to a log, or replay them — the basis of job queues and transactional systems. (This demo's `TaskQueue` holds one pending task at a time; a real scheduler would hold a `List` / `Deque` of them.)
- **Lambdas.** `IActuatorCommand` is a single-method (functional) interface, so `taskQueue.assignTask(joint::engage)` works identically — a method reference is a lightweight command.
- **Plain `main`, no Spring** — the pattern is pure OO structure and needs no container.

Relationship to the other Behavioral patterns

- **Command (this module)** — package a request as an object so it can be stored, passed, queued, and undone; decouples the invoker from the receiver.
- **Memento** — commonly paired with Command to implement `undo()` (the command snapshots the receiver's prior state).
- **Strategy** — also wraps behavior in an object, but represents *how to do* one interchangeable algorithm, not *a request to be dispatched/queued/undone* on a receiver.
- **Chain of Responsibility** — passes one request along handlers until one handles it; Command instead hands a fully-bound request to a single invoker to fire.

Reach for Command when you want to **parameterize objects with actions**, **queue or schedule** requests, support **undo/redo or macros**, or simply decouple the object that triggers an operation from the object that performs it.